

ui.line_plot 全面详解

`ui.line_plot` 是 NiceGUI 框架中专为**实时折线图可视化**设计的高阶组件，基于 Matplotlib/pyplot 封装，核心优势是简化动态数据更新流程，支持多线条渲染、数据点数量限制、批量更新优化等功能，尤其适配实时监控、数据流展示等场景。无需手动管理 Matplotlib 图表对象，通过简洁的 API 即可实现高性能的实时折线图渲染。以下从核心特性、使用场景、详细配置、高级技巧等维度展开全面解析。

一、核心概述

1. 本质与定位

- 专为**动态折线图**设计，封装了 pyplot 的折线图逻辑，聚焦实时数据更新场景（如传感器数据、系统监控、时序数据展示）。
- 内置数据点限制、批量更新、多线条支持等特性，避免重复编写模板代码，兼顾性能与易用性。
- 继承 NiceGUI 组件通用能力，支持样式定制、事件监听、可见性绑定，无缝融入 Web 界面。

2. 核心参数（初始化时必须填 / 常用）

参数名	类型	说明
<code>n</code>	int	线条数量（必填），需与后续 <code>push</code> 方法的 <code>y</code> 参数格式匹配。
<code>limit</code>	int	每条线的最大数据点数量，超过后自动移除最旧数据（默认无限制，建议实时场景设置合理值）。
<code>update_every</code>	int	批量更新阈值，累计推送 <code>update_every</code> 次数据后才刷新图表，减少 CPU 与带宽消耗（默认 1）。
<code>close</code>	bool	退出上下文后是否关闭图表（默认 <code>True</code> ，动态更新场景需设为 <code>False</code> ）。
<code>**kwargs</code>	关键字参数	传递给 <code>pyplot.figure</code> 的配置，如 <code>figsize</code> （图表尺寸）、 <code>dpi</code> （分辨率）等。
<code>style</code>	Style[Self]	图表容器样式，支持 Tailwind CSS（如 <code>bg-gray-50</code> 、 <code>rounded-lg</code> ）。
<code>classes</code>	Classes[Self]	图表容器的 HTML 类名，用于样式复用。

二、基础使用场景与示例

1. 基础实时折线图（多线条）

通过 `ui.timer` 定时生成数据，调用 `push` 方法推送数据到图表，自动渲染多线条实时更新效果。

示例：双线条实时正弦 / 余弦曲线

```
from nicegui import ui
import math
from datetime import datetime
```

```

# 初始化: 2条线、最大20个数据点、图表尺寸(3,2)、每5次推送更新一次
line_plot = ui.line_plot(
    n=2,
    limit=20,
    figsize=(8, 4),
    update_every=5
).with_legend(['sin(x)', 'cos(x)'], loc='upper right') # 添加图例

# 定义数据更新函数
def update_data():
    now = datetime.now()
    x = now.timestamp() # x轴: 时间戳 (也可直接用索引、时间字符串)
    y1 = math.sin(x)     # 第一条线数据
    y2 = math.cos(x)     # 第二条线数据
    # 推送数据: x为x轴列表, Y为二维列表 (每行对应一条线的Y值), 固定Y轴范围(-1.5,1.5)
    line_plot.push(
        x=[x],
        Y=[[y1], [y2]],
        y_limits=(-1.5, 1.5)
    )

# 定时器: 每0.1秒更新一次数据 (默认未激活)
timer = ui.timer(0.1, update_data, active=False)
# 复选框控制定时器激活/暂停 (绑定timer的active属性)
ui.checkbox('启动实时更新', value=False).bind_value(timer, 'active')

ui.run()

```

2. 单线条实时数据监控

设置 `n=1` 实现单线条渲染, 适用于单一指标监控 (如 CPU 使用率、温度变化)。

示例: 单线条随机数据监控

```

from nicegui import ui
import random

# 初始化: 1条线、最大30个数据点、图表尺寸(6,3)
line_plot = ui.line_plot(
    n=1,
    limit=30,
    figsize=(6, 3),
    style='bg-white rounded-lg shadow-sm p-2'
).with_legend(['随机数据'], loc='upper left')

# 定时生成随机数据 (0-100)
def update_single_line():
    x = len(line_plot._x_data) # x轴用数据点索引 (简化时序展示)
    y = random.randint(0, 100)
    line_plot.push(
        x=[x],
        Y=[[y]],

```

```

        y_limits=(0, 100) # Y轴固定范围0-100
    )

# 启动定时器（每0.5秒更新一次）
ui.timer(0.5, update_single_line, active=True)

ui.run()

```

3. 自定义图例与图表样式

通过 `with_legend` 方法配置图例位置、列数等，结合 `style` 参数美化图表容器。

示例：定制图例与样式

```

from nicegui import ui
import math
from datetime import datetime

# 初始化图表：3条线、最大25个数据点、每3次推送更新
line_plot = ui.line_plot(
    n=3,
    limit=25,
    figsize=(8, 4),
    update_every=3,
    style='bg-gray-50 rounded-xl shadow-md p-4' # 容器样式：灰色背景、圆角、阴影、内边距
)

# 配置图例：3列、居中显示、字体大小10
line_plot.with_legend(
    titles=['信号1', '信号2', '信号3'],
    loc='upper center',
    ncol=3,
    fontsize=10
)

# 生成3条不同振幅的正弦曲线
def update_three_lines():
    now = datetime.now().timestamp()
    x = now
    y1 = math.sin(x) * 1.2
    y2 = math.sin(x + 1) * 0.8
    y3 = math.sin(x + 2) * 1.5
    line_plot.push(
        x=[x],
        y=[[y1], [y2], [y3]],
        y_limits=(-2, 2)
    )

ui.timer(0.2, update_three_lines, active=True)
ui.run()

```

4. 动态控制数据更新（启动 / 暂停 / 清空）

结合按钮组件控制定时器激活状态，调用 `clear` 方法清空图表数据，适配交互场景。

示例：带控制按钮的实时折线图

```
from nicegui import ui
import random

# 初始化图表：1条线、最大50个数据点
line_plot = ui.line_plot(n=1, limit=50, figsize=(8, 4))
line_plot.with_legend(['实时指标'], loc='upper right')

# 定时器（初始未激活）
timer = ui.timer(0.3, lambda: line_plot.push(
    x=[len(line_plot._x_data)],
    y=[[random.uniform(0, 50)]]
), active=False)

# 控制按钮组
ui.row([
    ui.button('启动', on_click=lambda: setattr(timer, 'active', True)),
    ui.button('暂停', on_click=lambda: setattr(timer, 'active', False)),
    ui.button('清空', on_click=line_plot.clear) # 清空图表数据
])

ui.run()
```

5. 自定义坐标轴范围

通过 `push` 方法的 `x_limits` 和 `y_limits` 参数，固定或自动适配坐标轴范围，避免图表抖动。

示例：固定 Y 轴范围，X 轴自动适配

```
from nicegui import ui
import math
from datetime import datetime

line_plot = ui.line_plot(n=1, limit=20, figsize=(8, 3))
line_plot.with_legend(['波动数据'], loc='upper right')

def update_with_fixed_limits():
    now = datetime.now().timestamp()
    # 生成0-100的波动数据
    y = (math.sin(now) * 30) + 50
    line_plot.push(
        x=[now],
        y=[[y]],
        x_limits='auto', # X轴自动适配数据范围
        y_limits=(0, 100) # 固定Y轴0-100，避免抖动
    )
```

```
ui.timer(0.1, update_with_fixed_limits, active=True)
ui.run()
```

6. 结合实际数据来源（模拟传感器）

模拟传感器周期性采集数据，通过 `push` 方法实时推送，适配工业监控、环境监测等场景。

示例：模拟温度传感器数据

```
from nicegui import ui
import random
from datetime import datetime

# 模拟温度传感器：生成25-35℃的随机数据，含微小波动
def get_temperature():
    base_temp = 30
    波动 = random.uniform(-2, 2)
    return base_temp + 波动

# 初始化图表：1条线、最大30个数据点、每2次推送更新
line_plot = ui.line_plot(
    n=1,
    limit=30,
    update_every=2,
    figsize=(8, 4),
    style='bg-white rounded-lg p-3'
).with_legend(['温度 (℃)'], loc='upper left')

# 定时采集并推送数据
def update_temperature():
    now = datetime.now().strftime('%H:%M:%S') # X轴显示时间字符串
    temp = get_temperature()
    line_plot.push(
        x=[now],
        Y=[[temp]],
        y_limits=(25, 35) # 温度范围固定25-35℃
    )

# 每1秒更新一次（模拟传感器采样频率）
ui.timer(1.0, update_temperature, active=True)

ui.run()
```

三、核心属性与方法详解

1. 常用属性

属性名	类型	说明
<code>_x_data</code>	list	内部存储的 X 轴数据列表（只读，可用于调试数据长度）。

属性名	类型	说明
<code>_y_data</code>	<code>list[list]</code>	内部存储的 Y 轴数据列表（二维列表，每行对应一条线，只读）。
<code>visible</code>	<code>BindableProperty</code>	控制图表是否可见（支持绑定，如 <code>bind_visibility</code> ）。
<code>style</code>	<code>Style[Self]</code>	图表容器样式，支持动态修改（如 <code>line_plot.style('bg-blue-50')</code> ）。
<code>html_id</code>	<code>str</code>	图表在 HTML DOM 中的唯一 ID（v2.16.0+ 支持）。

2. 核心方法

方法名	作用	参数说明
<code>push(x, Y, x_limits='auto', y_limits='auto')</code>	推送新数据到图表（核心方法）。	- <code>x</code> : X 轴数据列表（长度需与每条线的新 Y 值数量一致，如单次推送 1 个点则为 <code>[x_val]</code> ）； - <code>Y</code> : 二维列表， <code>Y[i]</code> 为第 <code>i</code> 条线的新 Y 值列表（如 2 条线单次推送 1 个点则为 <code>[[y1], [y2]]</code> ）； - <code>x_limits</code> : X 轴范围（ <code>(min, max) / 'auto' / None</code> ，v2.10.0+）； - <code>y_limits</code> : Y 轴范围（同 <code>x_limits</code> ）。
<code>with_legend(titles, **kwargs)</code>	为图表添加图例。	- <code>titles</code> : 图例名称列表（长度需与 <code>n</code> 一致）； - <code>**kwargs</code> : 传递给 <code>pyplot.legend</code> 的参数（如 <code>loc</code> 位置、 <code>ncol</code> 列数、 <code>fontsize</code> 字体大小）。
<code>clear()</code>	清空图表所有数据点，重置折线图。	-
<code>update()</code>	手动触发图表刷新（默认无需调用， <code>push</code> 会自动根据 <code>update_every</code> 触发）。	-
<code>on(type, handler)</code>	绑定 Web 事件（如 <code>click</code> 、 <code>mousemove</code> ），实现图表交互。	<code>type</code> : 事件类型； <code>handler</code> : 回调函数，接收事件参数 <code>e</code> （含屏幕坐标等）。
<code>bind_visibility(target)</code>	绑定图表可见性到目标对象（如开关、复选框）。	<code>target_object</code> : 目标对象； <code>target_name</code> : 属性名（默认 <code>visible</code> ）。
<code>delete()</code>	删除图表组件及内部数据，释放资源。	-

四、高级技巧与注意事项

1. 性能优化

- 合理设置 `limit`：实时场景建议设 `limit=50-200`，避免数据点过多导致图表渲染卡顿、内存占用过大。
- 巧用 `update_every`：高频数据（如每秒推送 10 次）可设 `update_every=5-10`，减少页面刷新次数，降低 CPU 负载。
- 坐标轴范围优化：固定 `y_limits`（如温度、电压等有明确范围的指标），避免图表频繁缩放导致的视觉抖动与性能消耗。

2. 数据格式规范

- `x` 与 `y` 长度匹配：单次推送时，`x` 列表长度需等于每条线的新 `y` 值数量（如单次推 1 个点，`x=[x_val]`，`y=[[y1], [y2]]`；推多个点，`x=[x1,x2]`，`y=[[y1a,y1b], [y2a,y2b]]`）。
- `y` 维度匹配 `n`：`y` 必须是二维列表，长度等于初始化时的 `n`（如 `n=3`，`y` 需为 `[[y1], [y2], [y3]]`）。

3. 样式定制技巧

- 容器样式：通过 `style` 参数添加 Tailwind CSS 实现响应式布局（如 `style='width: 100%; max-width: 1000px'`），适配不同屏幕。
- 图表内部样式：若需修改线条颜色、粗细等，可通过 `pyplot` 上下文调整（需结合 `close=False`）：

```
with line_plot._figure as fig:
    ax = fig.gca()
    ax.lines[0].set_color('#b687ac') # 第一条线设为紫色
    ax.lines[1].set_linewidth(3)      # 第二条线加粗
```

4. 兼容性与版本说明

- 依赖版本：要求 NiceGUI ≥ 2.0.0，`x_limits/y_limits` 参数需 v2.10.0+ 支持，`html_id` 需 v2.16.0+。
- 动态更新注意：初始化时需设 `close=False`（默认 `True`），否则退出上下文后图表无法再更新。
- 图例限制：`with_legend` 的 `titles` 长度必须与 `n` 一致，否则图例显示异常。

5. 调试技巧

- 数据长度调试：打印 `len(line_plot._x_data)` 或 `len(line_plot._y_data[0])`，确认数据点数量是否符合 `limit` 配置。
- 推送频率调试：通过 `update_every` 调整后，可打印日志确认图表实际刷新频率，平衡实时性与性能。
- 样式问题排查：若图表不显示，检查 `push` 方法的 `x/y` 格式是否正确、`n` 是否与 `y` 维度匹配、`active` 是否为 `True`。

五、与其他图表组件的区别

`ui.line_plot` 是 `ui.pyplot/ui.matplotlib` 的**专用子集**，聚焦实时折线图场景，与通用组件的核心区别如下：

组件	核心优势	适用场景
<code>ui.line_plot</code>	简化实时折线图逻辑，支持批量更新、数据点限制	时序数据监控、实时数据流展示、多线条动态更新
<code>ui.pyplot</code>	通用 Matplotlib 封装，支持所有图表类型	静态图表、复杂定制化图表、复用现有 pyplot 代码
<code>ui.matplotlib</code>	上下文管理器优先，简洁的 Figure 操作	新开发的 Web 可视化项目，追求代码简洁性

六、总结

`ui.line_plot` 是 NiceGUI 中针对**实时折线图**场景的最优解，核心优势在于：

- 1. 专注动态更新：内置数据点限制、批量更新等特性，无需手动处理数据管理与渲染优化。
- 2. 易用性强：通过 `push` 方法一键推送数据，`with_legend` 快速配置图例，减少模板代码。
- 3. 性能均衡：通过 `limit` 和 `update_every` 平衡实时性与资源消耗，适配高频数据场景。
- 4. 生态融合：支持样式定制、事件监听、可见性绑定，无缝融入 Web 应用界面。

适用于传感器监控、系统性能实时展示、时序数据可视化、工业控制界面等场景，是 Python 开发者快速构建实时折线图的高效工具。